



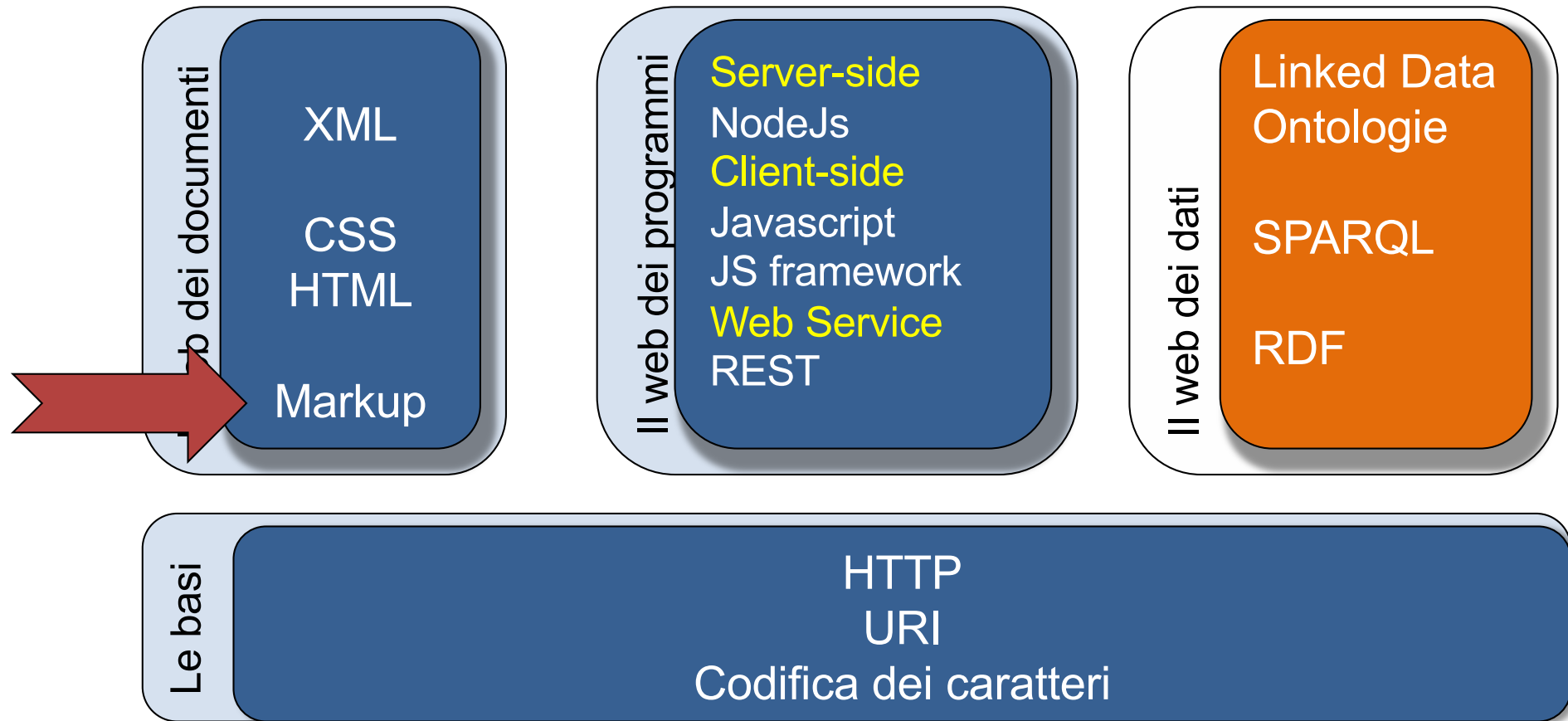
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Markup

Fabio Vitali

Corso di laurea in Informatica
Alma Mater – Università di Bologna

Argomenti delle lezioni



«Quando io uso una parola», disse Humpty Dumpty in tono piuttosto sprezzante, «significa quello che io scelgo che significhi - né più, né meno.»

Lewis Carroll

“Attraverso lo specchio”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduzione

Qui esaminiamo:

- Cos'è il markup
- Una storia del markup su computer
- Alcuni linguaggi di markup
- SGML e XML





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Rappresentazione dell'informazione

Dati

I dati sono la quantificazione delle informazioni raccolte e semplificate per un facile utilizzo da parte di applicazioni.

Da questo punto di vista, i dati non esistono in natura. Esiste sempre un attore (umano o automatico) che seleziona i dati e li rende in forma quantificata per la gestione automatica.

I dati vengono utilizzati dai programmi per fornire un'applicazione concreta ad algoritmi generici. Ad esempio:

- un programma che calcola l'area di un triangolo utilizza due parametri, la base e l'altezza.
- Un'esecuzione concreta del programma richiede dati effettivi per riempire i ruoli di base e altezze, ad esempio, $\text{area}(3,6) = 9$.



Strutture di dati

In molti casi (la maggior parte?), i dati sono in realtà complessi e vengono organizzati in strutture dati, che rendono alcuni usi dei dati molto più facili che se non fossero stati strutturati.

Le strutture dati rendono esplicite e sfruttabili le regolarità nei dati.

Ciò è particolarmente vero se non si dispone di singoli valori, ma di una grande quantità di valori relativi a molteplici entità che devono essere gestiti dal programma nel suo complesso

Ad esempio: tutti i miei contatti, tutte le mie fatture, tutti i miei libri, tutte le mie leggi, etc.

La maggior parte dei dati interessanti da gestire vengono raccolti e organizzati in collezioni di dati strutturati.



Valori, chiavi, entità

"42"

Questo è un dato. Non significa nulla: è un numero civico? un'età? Un numero di scarpe? Un dato di per sé non ha scopo.

"age: 42"

Questa è un'informazione. È composto da un **valore** (42) e da una **chiave** ("età") che fornisce contesto e comprensione al valore.

Le chiavi sono anche chiamate **proprietà, campi, etichette**, ecc. A volte il valore è un dato puro, a volte è un riferimento a qualcosa di più complesso.

```
{  
  name: "Mario",  
  surname: "Rossi",  
  age: 42,  
  city: "Bologna"  
}
```

Questa è una rappresentazione di un'entità della vita reale. Contiene diverse coppie chiave-valore che corrispondono a ciò che sappiamo dell'entità.

Le entità sono anche chiamate **record**.



Record

Un record è una collezione (una *tupla*) di elementi di dati etichettati, selezionati per il fatto che descrivono (***completamente?***) una singola entità. Rappresentano tutto ciò che si sa (***o che è rilevante sapere?***) su un'entità. La struttura dei record è l'astrazione della collezione di label (= ***chiavi***) dei valori necessari per riconoscere e descrivere ogni individuo di una collezione di dati.

<i>Label</i>	Value
<i>Name</i>	Mario
<i>Surname</i>	Rossi
<i>Birthdate</i>	1980-12-10
<i>Birthplace</i>	Bologna
...	...



Liste

A

a/an (det)	activity (n)	agency (n)
ability (n)	actor (n)	ago (adv)
able (adj)	actress (n)	agree (v)
• be able to	actually (adv)	ahead (adv)
about (adv & prep)	• She seems a bit strict at first, but she's actually very nice.	aim (n & v)
• about 500 students (adv)	• Are you actually going to take the job?	air (n)
• The film is about a small boy. (prep)	ad (advertisement) (n)	air conditioning (n)
above (adj, adv & prep)	add (v)	air force (n)
abroad (adv)	addition (n)	airline (n)
absent (adj)	• in addition	airport (n)
absolutely (adv)	address (n)	alarm (n)
• The movie was absolutely awful.	admire (v)	alarm clock (n)
accent(n)	admission (n)	album (n)
• She has a beautiful French accent.	• charges/cost/price	alike (adv)
accept (v)	admit (v)	alive (adj)
access (n)	adult (adj & n)	all (adj, adv, det & pron)
• disabled access	advance (n)	all right/alright (adj,adv & exclam)
• internet access	• book in advance	allow (v)
accident (n)	advanced (adj)	almost (adv)
accommodation (n)	advantage (n)	alone (adv & adj)
accompany (v)	adventure (n)	along (adv & prep)
according to (prep phr)	advert (n)	aloud (adv)
account (n)	advertise (v)	alphabet (n)
accountant (n)	advertisement (n)	already (adv)
accurate (adj)	advice (n)	also (adv)
ache (n)	advise (v)	although (conj)
achieve (v)	aeroplane (n)	altogether (adv)
across (adv & prep)	afford (v)	always (adv)
act (n & v)	afraid (adj)	a.m (adv)
• in the second act (of the play) (n)	after (adv,conj & prep)	amazed (adj)
• to act in a play (v)	afternoon (n)	amazing (adj)
• to act strangely (v)	afterwards (adv)	ambition (n)
action (n)	again (adv)	ambulance (n)
active (adj)	against (prep)	among (amongst) (prep)
	age (n)	amount (n)
	aged(adj)	amusing (adj)
		an (det)
		ancient (adj)

- Una lista è una sequenza di dati, ognuno dei quali fa riferimento, menziona o esprime un valore per alcune entità.
- Gli elenchi sono spesso ordinati in base ad un criterio preciso (spesso ma non necessariamente l'ordine alfabetico).
- I valori sono spesso considerati in modo atomico, anche se (come nell'esempio), la sintassi, la tipografia e altri trucchi possono rendere visibile l'esistenza di un'organizzazione implicita delle informazioni disponibili.



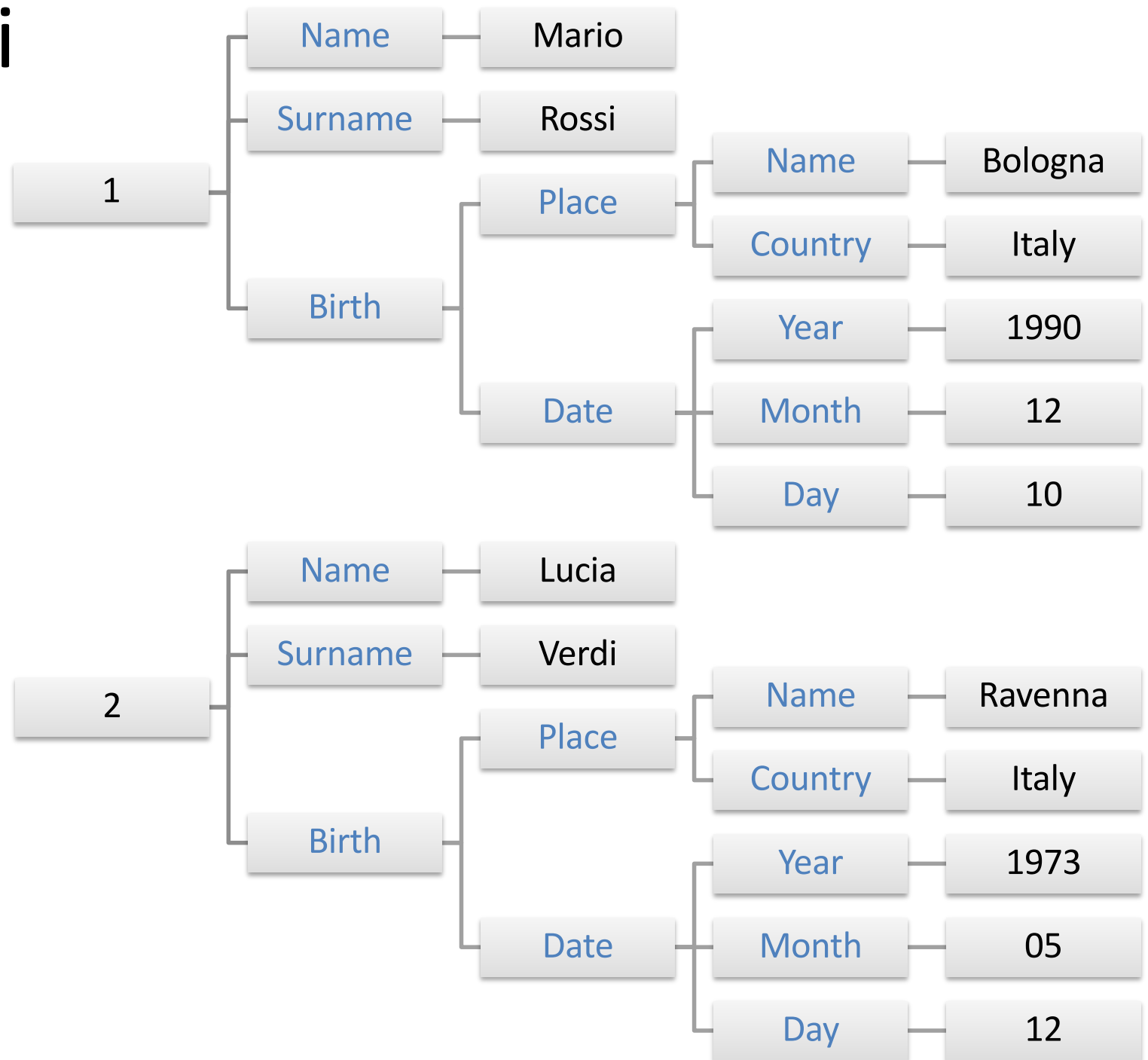
Tabelle

<i>Name</i>	Surname	Birthdate	Birthplace
Mario	Rossi	1990-12-10	Bologna (I)
Lucia	Verdi	1973-05-12	Ravenna (I)
John	Smith	2006-02-27	New York (US)
Mary	Brown	2002-09-07	London (UK)
...			

- Le tabelle sono un modo per rappresentare elenchi di dati strutturati in modo omogeneo (ad esempio, molti record con la stessa struttura).
- Ogni riga contiene gli elementi di dati di un singolo record, che rappresenta un'entità.
- Ogni colonna contiene i valori dello stesso campo (chiave) per ogni record della collezione.



Alberi



Alberi

Gli alberi sono strutture dati per rappresentare gerarchie nelle collezioni.

Le due forme più comuni di gerarchia sono contenimento e dominio.

- Gli eserciti sono gerarchie di dominio. La catena di comando in un'azienda è spesso una gerarchia di dominio.
- Gli elementi geometrici in uno schermo video sono spesso una gerarchia di contenimento. Gli elementi strutturali di un documento sono spesso una gerarchia di contenimento.

La gerarchia può essere trovata tra gli elementi di dati che descrivono una singola entità...

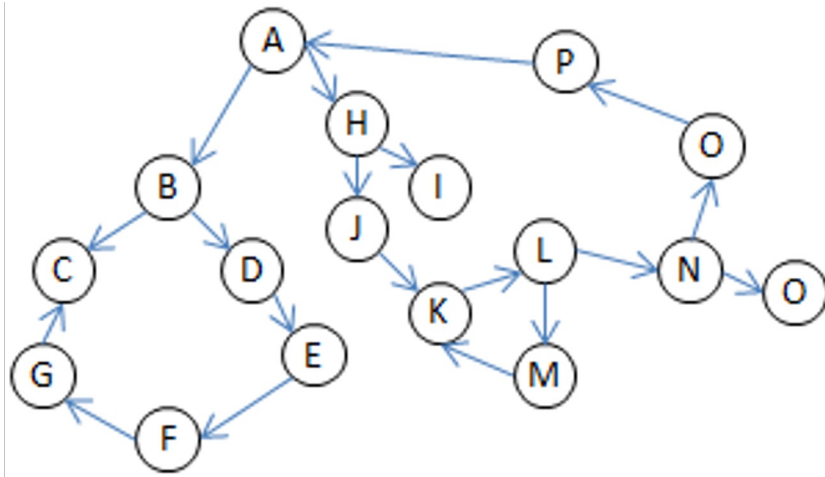
- Ad esempio: un libro contiene capitoli che contengono sezioni che contengono sottosezioni che contengono paragrafi che contengono testo.
- Queste entità non possono più essere descritte da un record

... o tra le entità descritte dalla collezione

- Ad esempio: un supervisore comanda molti task manager che comandano molti team leader che comandano molti dipendenti di livello base.
- La collezione che esprime i rapporti di comando (nell'esempio, tra i dipendenti) non può più essere descritta da una tabella.



Grafi



I grafi sono strutture di dati generiche per la rappresentazione di reti.

I **nodi** (dati) sono collegati tra loro da **archi** (relazioni).

I grafi possono essere utilizzati per rappresentare entità correlate tra loro...

- I nodi rappresentano gli elementi di dati e gli archi rappresentano le relazioni tra gli elementi di un'entità.
- Ad esempio, un composto chimico è descritto dai suoi atomi (i suoi nodi) e dalla struttura delle loro disposizioni (chi è legato a chi da quale tipo di legame)

... o per rappresentare collezioni

- I nodi rappresentano le singole entità e gli archi rappresentano le relazioni tra le entità
- Ad esempio, le persone in una grande organizzazione, dove si intende mostrare non solo la relazione di comando, ma tutti i tipi di relazioni, inclusi parentela, amicizia, inimicizia, provenienza, ecc.



Testo

Il testo è un po' complicato.

Il testo non è un dato, non è una struttura di dati, non è una collezione di dati.

- Il testo non è dato perché è stato inventato prima e indipendentemente dalla quantificazione e dalla strutturazione dei dati.

Il testo non è un record, non è organizzato in tabelle, alberi o grafici.

Il testo può contenere dati, nella misura in cui c'è un attore (umano o automatico) che identifica e qualifica parte del testo come portatore di un significato proprio che vale la pena quantificare (ad esempio, trasformarsi in dati).



Testo

Alcune cose sul testo:

- Il testo ha una struttura (struttura di contenimento, per la precisione);
- Il testo ha un ordine (alcune parti vengono prima di altre);
- Il testo ha parti che sono significative e importanti da identificare e altre parti che non lo sono.

L'importanza è negli occhi dell'osservatore e dipende dall'uso del testo (*non c'è solo visualizzazione!*).

La teoria del markup descrive i testi come un albero ordinato ed etichettato di elementi e nodi di testo.

- La struttura di contenimento è completamente descritta dall'albero, ma gli elementi all'interno della gerarchia sono in un ordine che non può essere ignorato.
- I nodi debbono essere etichettati in modo esplicito.
- Gli elementi possono contenere altri elementi o nodi di testo o una raccolta di elementi e nodi di testo (contenuto misto)



Ricapitolando...

- Sia gli elementi di dati che le collezioni possono assumere una forma a record, tabella, albero, grafo indipendentemente l'uno dall'altro.
- È importante distinguere quando la struttura dei dati si applica agli elementi di dati o alle collezioni di dati.
- È possibile e frequente avere tabelle di record, tabelle di alberi, tabelle di grafi, alberi di record, alberi di tabelle, alberi di grafi, grafi di record, grafi di tabelle, grafi di alberi, ecc.
- I dati e le collezioni di dati non devono essere confusi.
- I documenti di testo hanno le loro peculiarità e non possono essere gestiti allo stesso modo dei dati.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Formati per strutture dati

Una breve lista di formati dati

Tabelle

- CSV, Tab-delimited
- Excel, Libre office, etc.

Alberi

- JSON
- YAML

Grafi

- RDF (Turtle)

Testo

- Plain text
- TROFF, NROFF
- TeX, LaTeX
- Markdown
- HTML, XML

Immagini

- *Raster*
 - *Lossless: TIFF, GIF, PNG*
 - *Lossy: JPG*
- *Vettoriale: SVG*

Contenitori (metafile)

- *Archivi (Zip)*
- *Pacchetti (Docx, ODF, PKG, MSI)*
- *Audio (WAV, AIFF)*
- *Multimedia (AVI, MPG)*



Tabelle: CSV (Comma Separated Values)

```
Year,Make,Model,Description,Price
1997,Ford,E350,"ac, abs, moon",3000.00
1999,Chevy,"Venture ""Extended Edition""",4900.00
1999,Chevy,"Venture ""Extended Edition, Very Large""",5000.00
1996,Jeep,Grand Cherokee,"MUST SELL!
air, moon roof, loaded",4799.00
```

CSV (Comma-separated values)

- Un vecchio formato per lo scambio di dati tabulari.
- Ogni riga è un record, ogni elemento è il valore per un campo.
- La prima riga è un elenco di tutte i nomi dei campi delle altre righe
- Utilizza un ritorno a capo per le righe
- Utilizza una virgola, un punto e virgola o una barra verticale | come separatore di campo.
- Queste scelte limitano l'uso dei caratteri di ritorno a capo e virgole nel testo.
- Dati tabellari: abbastanza buoni, purché regolari e strutturati.
- Dati gerarchici: nessun supporto.
- Testo: solo testo normale e in brevi frammenti.



Tabelle: Tab-delimited values

Year	Make	Model	Description	Price
1997	Ford	E350	"ac, abs, moon"	3000.00
1999	Chevy	Venture	"Extended Edition"	4900.00
1999	Chevy	Venture	"Extended Edition, Very Large"	5000.00
1996	Jeep	Grand Cherokee	"air, moon roof"	4799.00

- E' la stessa idea di CSV, ma utilizza i caratteri di tabulazione come separatori di campo
- Ogni riga è un record, ogni elemento è il valore per un campo.
- Come in CSV, la prima riga è un elenco di tutte le chiavi per le righe
- Utilizza un ritorno a capo per le righe
- Utilizza un carattere di tabulazione come separatore di campo.
- Queste scelte limitano l'uso o i caratteri di ritorno a capo e le tabulazioni nel testo.
- Stessi vantaggi e svantaggi del CSV



Tabelle: XLSX, ODF

AutoSave OFF Book1.xlsx

Home Insert Draw Page Layout Formulas Data Review View Tell me

D12

	A	B	C	D	E
1	Year	Make	Model	Description	Price
2	1997	Ford	E350	ac abs moon	3000
3	1999	Chevy	Venture "Extended Edition"		4900
4	1999	Chevy	Venture "Extended Edition Very Large"		5000
5	1996	Jeep	Grand Cherokee	MUST SELL! air moon roof loaded	4799

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<sst xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main" count="15" uniqueCount="14">
  <si>
    <t>Year</t>
  </si>
  <si>
    <t>Make</t>
  </si>
  <si>
    <t>Model</t>
  </si>
  <si>
    <t>Description</t>
  </si>
  <si>
    <t>Price</t>
  </si>
  <si>
    <t>Ford</t>
  </si>
  <si>
    <t>E350</t>
  </si>
  <si>
    <t>ac abs moon</t>
  </si>
  <si>
    <t>Chevy</t>
  </si>
  <si>
    <t>Venture "Extended Edition"
    </t>
  </si>
  <si>
    <t>Venture "Extended Edition Very Large"
    </t>
  </si>
  <si>
    <t>Grand Cherokee</t>
  </si>
  <si>
    <t>MUST SELL! air moon roof loaded</t>
  </si>
</sst>
```

```
<row r="2" spans="1:5" x14ac:dyDescent="0.2">
  <c r="A2" s="5">
    <v>1997</v>
  </c>
  <c r="B2" s="3" t="s">
    <v>5</v>
  </c>
  <c r="C2" s="3" t="s">
    <v>6</v>
  </c>
  <c r="D2" s="4" t="s">
    <v>7</v>
  </c>
  <c r="E2" s="3">
    <v>3000</v>
  </c>
</row>
```

- XLSX è il formato ufficiale per Microsoft Excel. ODF è il formato ufficiale per Open Office, Apache Office, ecc.
- Entrambi sono contenitori di molti file diversi, per il contenuto, le relazioni con altri file, le stringhe condivise e i valori effettivi.
- I file "sheetN.xml" sono file XML contenenti, uno per ogni cella, sia i valori numerici che i riferimenti alle stringhe condivise.



Alberi: JSON

```
[
  {
    "year": "1997",
    "type": {
      "make": "Ford",
      "model": "E350"
    },
    "features": [ "ac", "abs", "moon" ],
    "price": 2000.00
  }, {
    "year": "1999",
    "type": {
      "make": "Chevy",
      "model": "Venture \"Extended\""
    },
    "price": 4900.00
  }, {
    "year": "1996",
    "type": {
      "make": "Jeep",
      "model": "Grand Cherokee"
    },
    "features": [ "air", "moon roof" ],
    "description": "Loaded! MUST SELL!",
    "price": 4799.00
  }
]
```

JSON (*JavaScript Object Notation*)

- Nativo del linguaggio Javascript (utilizzato sul Web)
- Supporto in tutti i linguaggi di programmazione
- Le graffe e le quadre indicano contenitori (con differenze semantiche).
- Le coppie chiave-valore sono separate da due punti.
- I valori possono essere anche altre sotto-entità in una gerarchia.
- Dati tabellari: abbastanza buoni
- Dati gerarchici: abbastanza buoni
- Testo: solo testo semplice

Alberi: JSON

```
{
  "name": "Mario",
  "surname": "Rossi",
  "birth": {
    "place": {
      "city": "Bologna",
      "country": "Italy"
    },
    "date": {
      "year": 1990,
      "month": "December",
      "day": 10
    }
  },
  "postal address": {
    "street": {
      "name": "Via Indipendenza",
      "number": "15"
    },
    "city": "Bologna",
    "country": "Italia"
  },
  "telephone": [
    {
      "type": "home",
      "number": "051 123456"
    }, {
      "type": "cell",
      "number": "335 987654"
    }
  ]
}
```

JSON (*JavaScript Object Notation*)

- Nativo del linguaggio Javascript (utilizzato sul Web)
- Supporto in tutti i linguaggi di programmazione
- Le graffe e le quadre indicano contenitori (con differenze semantiche).
- Le coppie chiave-valore sono separate da due punti.
- I valori possono essere anche altre sotto-entità in una gerarchia.
- Dati tabellari: abbastanza buoni
- Dati gerarchici: abbastanza buoni
- Testo: solo testo semplice

Hierarchical data: YAML

```
name: Mario
surname: Rossi
# Annido i valori con spazi,
# I tab non sono ammessi
birth:
  place:
    city: "Bologna"
    country: "Italy"
  date:
    year: 1990
    month: "December"
    day: 10
postal address:
  street:
    name: "Via Indipendenza"
    number: 15
  city: "Bologna"
  country: "Italia"
telephone:
- type: "home"
  number: "051 123456"
- type: "cell"
  number: "335 987654"
```

YAML (Ain't a Markup Language) è un altro formato di dati gerarchico fortemente basato su JSON, ma ispirato alla sintassi Python:

- Superset di JSON (un file JSON è anche un file YAML)
- Spazi e rientro come modello di annidamento
- Supporta gli stessi valori e strutture di JSON
- Supporta i commenti
- Stessi vantaggi e svantaggi di JSON



Grafi: RDF

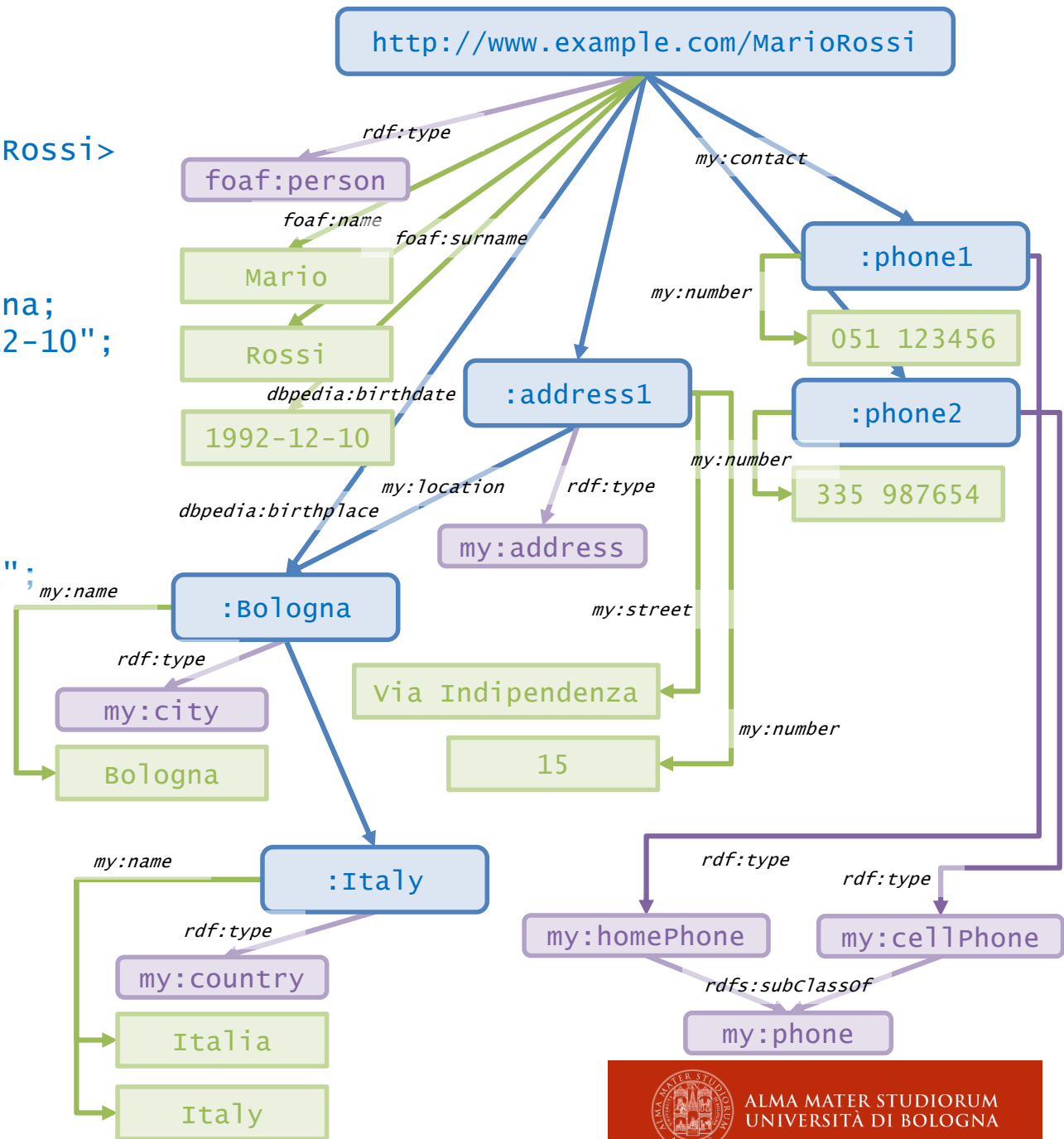
```
<http://www.example.com/MarioRossi>
  a foaf:person;
  foaf:name "Mario";
  foaf:surname "Rossi";
  dbpedia:birthplace :Bologna;
  dbpedia:birthdate "1992-12-10";
  my:address :address1;
  my:contact :phone1;
  my:contact :phone2.
```

```
:address1 a my:address;
  my:street "Via Indipendenza";
  my:number "15";
  my:location :Bologna.
```

```
:Bologna a my:city;
  my:name "Bologna";
  my:belongsTo :Italy.
```

```
:Italy a my:country;
  my:name "Italia"@it;
  my:name "Italy"@en;
```

```
:phone1 a my:homePhone;
  my:number "051 123456".
:phone2 a my:cellPhone;
  my:number "335 987654".
```



Grafi: RDF

```
<http://www.example.com/MarioRossi>  
  a foaf:person;  
  foaf:name "Mario";  
  foaf:surname "Rossi";  
  dbpedia:birthplace :Bologna;  
  dbpedia:birthdate "1990-12-10";  
  my:address :address1;  
  my:contact :phone1;  
  my:contact :phone2.
```

```
:address1 a my:address;  
  my:street "Via Indipendenza";  
  my:number "15";  
  my:location :Bologna.
```

```
:Bologna a my:city;  
  my:name "Bologna";  
  my:belongsTo :Italy.
```

```
:Italy a my:country;  
  my:name "Italia"@it;  
  my:name "Italy"@en;
```

```
:phone1 a my:homePhone;  
  my:number "051 123456".  
:phone2 a my:cellPhone;  
  my:number "335 987654"
```

RDF (Resource Description Framework)

- Non un formato, ma un modello concettuale in molti formati dati (qui è Turtle)
- Crea semplici affermazioni *soggetto predicato oggetto* sulle entità.
- Utilizza URI per soggetti e predicati e per oggetti che non sono semplici valori
- Le affermazioni con lo stesso soggetto possono essere raggruppate
- Poiché utilizza riferimenti a entità come oggetto dei predicati, le entità possono essere citate in molti luoghi (vedi [:Bologna](#))
- Può utilizzare vocabolari ([foaf:](#), [dbpedia:](#), [my:](#)) e regole per la validazione (ontologie)
- Le entità possono essere associate a classi per garantire omogeneità e proprietà attese.
- Dati tabulari: abbastanza buoni, ma totalmente frammentati
- Dati gerarchici: nessun supporto.
- Testo: solo testo semplice e breve.
- Riferimenti: supporto completo.

Testo: plain text

IS THIS A HEADER?

Use basic characters for everything. Paragraphs are separated by a CR, or LF, or CRLF.

No styles, no typography, no structure

Formato del testo utilizzando solo i caratteri della loro codifica per gli effetti speciali.

ASCII (e ISO Latin 1 e UTF-8) hanno 33 codici speciali per questo, ma di scarsa utilità nelle applicazioni moderne.

Carriage return (CR), Line feed (LF), spazio, tab e poco altro.

- Dati tabellari: non appropriato
- Dati gerarchici: non appropriato
- Testo: supporto completo per testo lungo
- Presentazione dei dati: solo la tipografia più semplice



Testo: Markdown

A first level header
=====

Paragraphs are separated by a blank line.

2nd paragraph. **Italic**, ****bold****, and ``monospace``.

Lists look like this:

- * this one
- * that one
- * the other one

Formato di testo che utilizza semplici trucchi nel testo per rappresentare strutture e tipografia.

- Dati: inappropriati
- Dati gerarchici: inappropriati
- Testo: supporto completo per testo lungo
- Presentazione dei dati: solo la tipografia più semplice
- Riferimenti: collegamenti ipertestuali



Testo: HTML

```
<html>
  <body>
    <h1>A first level header</h1>

    <p>Paragraphs are contained inside elements.</p>
    <p>2nd paragraph. <i>Italic</i>, <b>bold</b>, and
      <code>monospace</code>.</p>

    <p>Lists look like this:</p>
    <ul>
      <li>this one</li>
      <li>that one</li>
      <li>the other one</li>
    </ul>
  </body>
</html>
```

Attuale formato di punta per i documenti di testo sul Web

Il testo è organizzato in elementi racchiusi all'interno di tag. C'è un elenco di tag che costituisce il vocabolario HTML.

Utilizza linguaggi aggiuntivi (ad esempio, CSS e Javascript per tipografia, layout e interattività sofisticati

- Dati tabellari: rappresentati come testo (ad es. come <table>)
- Dati gerarchici: non naturalmente
- Testo: supporto completo per testo lungo
- Riferimenti: collegamenti ipertestuali



Testo: XML

```
<article xmlns='http://docbook.org/ns/docbook'>
  <section>
    <title>A first level header</title>

    <para>Paragraphs are contained inside elements.</para>
    <para>2nd paragraph. <emphasis>Italic</emphasis>,
      <emphasis role="bold">bold</emphasis>.</para>
  </section>
</article>
```

Un meta-linguaggio di markup per documenti di testo.

- Come l'HTML, il testo è organizzato in elementi racchiusi all'interno di tag.
- Non c'è una lista di tag, ma possiamo costruire la nostra lista (chiamata vocabolario o schema XML. In questo caso: docbook).
- Forte enfasi sul markup descrittivo, sulle gerarchie, sui vocabolari personalizzati.
- Dati: rappresentati come testo (ad es. in una tabella)
- Dati gerarchici: in modo molto naturale
- Testo: supporto completo per testo lungo
- Riferimenti: collegamenti ipertestuali





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Cos'è il markup

Cos'è il markup? (1)

Definiamo markup ogni mezzo per rendere esplicita una particolare interpretazione di un testo.

Per esempio, tutte quelle aggiunte al testo scritto che permettono di renderlo più fruibile.

Oltre a rendere il testo più leggibile, il markup permette anche di specificare ulteriori usi del testo.

Con il markup per sistemi informatici (il nostro caso), specifichiamo le modalità esatte di utilizzo del testo nel sistema stesso.

Il markup non è soltanto un inevitabile e sgradevole risultato della informatizzazione dell'arte tipografica. Non è qualcosa che sta con noi a causa dell'informatica.



Cos'è il markup? (2)

Quando un autore scrive, da millenni a questa parte, specifica anche i delimitatori di parola (chiamati spazi), i delimitatori di frase (chiamati virgole) e i delimitatori di periodo (chiamati punti).

La numerazione delle pagine o l'uso dei margini per creare effetti sul contenuto sono noti da centinaia di anni.

Eppure questo a stretto rigore non fa parte del testo, ma del markup: nessuno dirà ad alta voce 'virgola' o 'punto' nel leggere un testo, ma creerà adeguati comportamenti paralinguistici (espressioni, toni, pause) per migliorare in chi ascolta la comprensione del testo.



La piramide dei linguaggi di markup

Energia / Informazione



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Modi del markup: proprietario vs. pubblico

Un formato proprietario è stato creato da una specifica azienda con uno specifico scopo commerciale. L'azienda ne detiene i diritti, e dunque è in grado di modificarlo, aggiornarlo o rivoluzionarlo in qualunque momento e per qualunque motivo.

Un formato pubblico è stato creato da un gruppo di interesse (individui, aziende, enti non commerciali, ecc.) come modello di armonizzazione tra le esigenze di ciascun partecipante.

Il gruppo tipicamente pubblica le specifiche del formato, permettendo a chiunque di realizzare strumenti software per quel formato. A volte questo si concretizza in uno standard ufficiale, avente valore normativo.



Modi del markup: binario vs. leggibile

Un formato binario è la memorizzazione esatta delle strutture in memoria dell'applicazione, che niente hanno a che vedere con le esigenze di comprensione di esseri umani. Il testo non è visibile o è visibile per caso.

Un formato leggibile invece è fatto per essere, in casi speciali, letto anche da esseri umani, che possono intervenire per operazioni di emergenza.

L'applicazione deve trasformare quanto legge in una struttura interna utile per le operazioni di modifica o presentazione. Questa fase si chiama parsing.



Modi del markup: interno vs. esterno

Il markup interno inserisce istruzioni di presentazione all'interno del testo, in mezzo alle parole.

Il markup esterno prevede due blocchi di informazioni: il contenuto e il markup, separati e collegati da indirizione.

Il markup interno richiede sintassi particolari per distinguere il markup dal contenuto. Tipicamente si adottano segnalatori particolari che cambiano il tipo di interpretazione del documento. La presenza del carattere segnalatore nel testo richiede l'adozione di tecniche di escaping.

Il markup esterno richiede un meccanismo di indirizione, basato su indirizzi, offset o identificatori, per associare con correttezza il markup al contenuto.



Modi del markup: procedurale vs. descrittivo

Il markup assolve a diversi ruoli a seconda del sistema di elaborazione, dell'applicazione, dello scopo a cui il documento è soggetto.

- Puntuazionale
- Presentazionale
- Procedurale
- Descrittivo
- Referenziale
- Metamarkup



Markup puntuazionale

- Il markup puntuazionale consiste nell'usare un insieme prefissato di segni per fornire informazioni perlopiù sintattiche sul testo.
- Le regole di punteggiatura sono sostanzialmente stabili, note agli autori, e frequenti nei documenti. Per questo gli autori tipicamente forniscono il loro markup puntuazionale autonomamente.
- Esistono tuttavia notevoli problemi nell'uso della punteggiatura:
 - Incertezze strutturali (virgola, punto e virgola o punto?),
 - Incertezze grafiche (virgolette aperte e chiuse o neutre?),
 - ambiguità procedurali (il punto viene usato sia per segnare la fine di una frase, che l'esistenza di un'abbreviazione, senza contare i tre puntini di sospensione).



Markup presentazionale

- Il markup presentazionale consiste nell'indicare effetti (tipografici o altro) per rendere più chiara la presentazione del contenuto.
- Nel testo, possono essere cambi di paragrafo o di pagina, interlinea, pallini per liste, ecc.
- E' altresì markup presentazionale: cambiare pagina all'inizio di una nuova sezione, scrivere "Capitolo 3" in cima alla pagina, ecc.



Markup procedurale

- Il markup procedurale consiste nell'indicare con precisione ad un sistema automatico che effetto attivare e che procedura (serie di istruzioni) eseguire nella visualizzazione del contenuto.
- In definitiva, utilizzo le capacità del sistema di presentazione per avere con precisione l'effetto voluto.
- Esempio: *Wordstar Dot Commands*

.PL 66
.MT 6
.MB 9
.LH 12



Markup descrittivo

- Il markup descrittivo consiste nell'identificare strutturalmente il tipo di ogni elemento del contenuto.
- Invece di specificare effetti tipografici come l'allineamento o l'interlinea, ne individuo il ruolo all'interno del documento, specificando che un elemento è un titolo, un paragrafo, o una citazione.



Markup referenziale

- Il markup referenziale consiste nel fare riferimento ad entità esterne al documento per fornire significato o effetto grafico ad elementi del documento. Per esempio, utilizzare una sigla nota che venga poi sostituita dalla parola intera durante la stampa
- Es.: l'autore scrive “CdL” e il sistema trasforma automaticamente l'input in “Corso di Laurea”



Metamarkup

- Il metamarkup consiste nel fornire regole di interpretazione del markup e permette di estendere o controllare il significato del markup.
- Ad esempio, la possibilità di definire macro per l'interpretazione e la visualizzazione del documento, o il processo di definizione degli elementi e delle procedure valide di un documento.



Un testo su carta

Tre Uomini in Barca

Jerome K. Jerome

1889

Capitolo primo

Tre invalidi - Le sofferenze di George e Harris - La vittima di centosette malattie inguaribili - [...]

Eravamo in quattro: George, William Samuel Harris, e io, Montmorency. Standocene seduti in camera mia, fumavamo e parlavamo di quanto fossimo malridotti... malridotti, dal punto di vista della salute, intendo, naturalmente.

Ci sentivamo tutti piuttosto giù di corda, ...

Un testo senza markup

Questo è il testo completamente senza markup, come poteva essere scritto su un papiro della biblioteca di Alessandria, nel II o III secolo a.C.

treuominiinbarcajeromekjerome1889capitolop
rimotreinvalidilesofferenzedigeorgeeharris
lavittimadicentosettemalattieinguaribilier
avamoinquattrogeorgewilliamsamuelharriseio
montmorencystandocenesedutiincameramiafuma
vamoeparlavamodiquantofossimomalridottimal
ridottidalpuntodivistadellasaluteintendona
turalmentecisentivamotuttipiuttostogiùdico
rda



Markup metabolizzato

Aggiungiamo markup puntuazionale e presentazionale: maiuscole/minuscole, punteggiatura, spazi e ritorni a capo sono essi stessi elementi di markup.

```
Tre Uomini in Barca
Jerome K. Jerome (1889)
Capitolo primo
Tre invalidi - Le sofferenze di George e Harris - La
vittima di centosette malattie inguaribili - [...]
Eravamo in quattro: George, William Samuel Harris, e
io, Montmorency. Standocene seduti in camera mia,
fumavamo e parlavamo di quanto fossimo malridotti...
malridotti, dal punto di vista della salute, intendo,
naturalmente.
Ci sentivamo tutti piuttosto giù di corda, ...
```


Markup procedurale

Sono comandi, o istruzioni che il sistema di lettura (umano o elettronico) deve eseguire sul testo. Ad esempio, istruzioni su come andare a capo, come decidere i margini, ecc. Questo è RTF.

```
{\rtf1 \mac \ansicpg10000 \uc1 \pard \plain \s15 \qc \widctlpar
\adjustright \f4 \fs48 \cgrid {Tre Uomini in Barca \par } \pard \plain
\widctlpar \adjustright \f4 \cgrid { \line \par \par \par \par \par }
\pard \plain \s1 \qc \keepn \widctlpar \outlinelevel0 \adjustright \i
\f4 \fs36 \cgrid {Jerome K. Jerome \par } \pard \plain \qc \widctlpar
\adjustright \f4 \cgrid { \fs36 [...] \par 1889 \line \par } \pard
\widctlpar \adjustright { \page } { \b \fs36 Capitolo primo} { \par
\par \par \line } { \i Tre invalidi - Le sofferenze di George e Harris
- La vittima di centosette malattie inguaribili - [ \u8230 \c9] \par
} { \line } { \fs28 Eravamo in quattro: George, William Samuel Harris,
e io, Montmorency. Standocene seduti in camera mia, fumavamo e
parlavamo di quanto fossimo malridotti \u8230 \c9 malridotti, dal
punto di vista della salute, intendo, naturalmente. \line Ci sentivamo
tutti piuttosto gi \u249 \c9 di corda, ... \par }}
```

Markup descrittivo

Sono informazioni (descrizioni) sugli elementi del documenti, che ne specificano il ruolo, la giustificazione, la relazione con gli altri elementi.

```
<ROMANZO>
  <TITOLO>Tre Uomini in Barca</TITOLO>
  <AUTORE>Jerome K. Jerome</AUTORE>
  <ANNO>1889</ANNO>
  <CAPITOLO>
    <TITOLO>Capitolo primo</TITOLO>
    <INDICE>
      <EL>Tre invalidi</EL>
      <EL>Le sofferenze di George e Harris </EL>
      <EL> La vittima di centosette malattie inguaribili </EL>
    </INDICE>
    <PARA>Eravamo in quattro: George, William Samuel Harris, e io,
Montmorency. Standocene seduti in camera mia, fumavamo e parlavamo di
quanto fossimo malridotti... malridotti, dal punto di vista della
salute, intendo, naturalmente. </PARA>
    <PARA>Ci sentivamo tutti piuttosto giù di corda, ...</PARA>
  </CAPITOLO>...
</ROMANZO>
```



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

I linguaggi di markup

TROFF/NROFF (1)

Nato nel 1973, fa parte della distribuzione Unix standard. Sotto Linux si chiama Groff

E' ancora usato per documentazione tecnica, in particolare i manuali on-line di Unix

Esiste un formatter che è in grado di creare documenti stampabili sia su stampante (troff) che su schermo a carattere (nroff).

I comandi sono o esterni (compaiono su righe autonome precedute da un punto) o interni (introdotte dal carattere di escape "\")

Permette la definizione e l'uso di quattro tipi di carattere: roman, italic, bold e symbol.

Permette la creazione di macro complesse (il nome è al massimo di due caratteri, però)



TROFF/NROFF (2)

`.\ " Troff UTF-8`

`.po 4c`

`.ll 8c`

`.ad 1`

`.hy 14`

`.ps 10`

`.vs 12`

`.ft B`

`1. Introduzione a troff`

`.ft P`

`.sp`

Questo è un esempio di documento scritto in troff puro. Le righe vuote terminano i paragrafi.

`.ft B`

`1.1 Paragrafi`

`.ft P`

`.sp`

Il testo di un paragrafo termina quando nel sorgente compare una riga vuota.

`.ft B`

`1.2 Elenco puntato`

`.ft P`

`.sp`

`\- Primo punto`

`.br`

`\- Secondo punto`

`.br`

`\- Terzo punto`

.po 4c — margine sinistro (page offset) a 4 cm.

.ll 8c — larghezza riga (line length) a 8 cm.

.ad 1 — allineamento a sinistra (adjust left).

.hy 14 — modalità di sillabazione 14 (hyphenation attiva in modalità 14).

.ps 10 — corpo del font 10 pt. (point size)

.vs 12 — interlinea 12 pt (vertical spacing).

.ft B — seleziona il font grassetto.

.ft P — ripristina il font precedente (di solito Roman).

.sp — spazio verticale (una riga per default).

.br — interruzione di riga forzata.

\- — stampa un segno meno ad inizio riga.



TeX e LaTeX (1)

Realizzato a metà degli anni 70 da Donald Knuth (famoso informatico teorico).

Linguaggio di programmazione Turing-completo, dotato di comandi per la formattazione di testi (circa 300 comandi fondamentali, detti primitive).

Di notevole complessità, è rivolta unicamente a programmatori e tipografi molto sofisticati.

Metafont è un sistema associato a TeX per la descrizione delle forme dei caratteri di un font attraverso formule matematiche.

TeX permette la generazione di macro che semplificano notevolmente la generazione di testi particolarmente sofisticati.

Esistono librerie di macro che permettono di scrivere documenti arbitrariamente complessi: matematica, chimica, musica, grafica vettoriale, grafica bitmap, ecc. Il supporto per la notazione matematica è eccellente (e ne ha garantito la sopravvivenza fino ad oggi).

Nel 1985 Leslie Lamport sviluppò LaTeX, una raccolta di macro TeX per la generazione di una ventina circa di tipi di documento particolarmente comuni: articolo, libro, lettera, annuncio, ecc.



LaTeX (2)

```
\documentclass{article}
```

```
\setlength{\textwidth}{7cm}
```

```
\setlength{\textheight}{7cm}
```

```
\begin{document}
```

```
\section{Introduzione a TeX/LaTeX}
```

Questo è un esempio di documento scritto con LaTeX. Come si può vedere è già stato definito uno stile generale del documento: `\texttt{article}`.

```
\subsection{Gli ambienti}
```

LaTeX utilizza gli ambienti per definire comportamenti circoscritti a zone particolari del testo.

Per esempio, la centratura si ottiene utilizzando l'ambiente `\texttt{center}`.

```
\begin{center}
```

Questo è un esempio di testo centrato.

```
\end{center}
```

```
\end{document}
```



LaTeX e TeX (3)

LaTeX	TeX
<pre>\begin{itemize} \item Primo \item Secondo \item Terzo \end{itemize}</pre>	<pre>\newdimen\Itemwd \Itemwd=1.5em \def\itemize{\par\begingroup\parindent=0 pt\parskip=0pt \def\item{\par\hangindent=\Itemwd\hang after=1\noindent \hbox to \Itemwd{\hfil\$\bullet\$}\enskip}} \def\enditemize{\par\endgroup}</pre>

- **\newdimen**: alloca un nuovo registro di lunghezza (dimensione).
 - **\parindent**: rientro della prima riga dei paragrafi.
 - **\parskip**: spazio verticale tra paragrafi.
 - **\hangindent**: ampiezza del rientro “sporgente” (hanging indent).
 - **\hangafter**: quante righe iniziali non subiscono/oppure subiscono il rientro.
 - **\hbox to <dim>{...}**: scatola orizzontale di larghezza fissata; lo spazio interno si espande/contrae.
 - **\hfil**: collante orizzontale a elasticità “infinita” (si espande per riempire).
 - **\enskip**: spazio orizzontale pari a circa **0.5 em**.
 - **\$**: entra/esce dalla modalità matematica inline.
-

wiktext & markdown

Linguaggi wiki (wiktext/MediaWiki)

Nato a metà anni '90 con Ward Cunningham (WikiWikiWeb).

Markup leggero per testi collaborativi: titoli, elenchi, link, tabelle, immagini.

Poche regole, sintassi semplice.

Usato per documentazione e knowledge base; adatto a non programmatori.

Nel 2002 nasce **MediaWiki**, piattaforma di riferimento adottata da Wikipedia.

Markdown

Creato nel 2004 da John Gruber.

Markup leggero orientato alla leggibilità in testo puro.

Set ristretto di costrutti: titoli, enfasi, link, elenchi, code span, blocchi di codice.

E' estendibile: tabelle, footnote, task list, diagrammi, math (non nello standard).

Molte varianti: CommonMark e GitHub Flavored Markdown sono i più comuni.

Usato ovunque per README, note, blog, documentazione e static site generators.



wikitext & markdown (2)

An h1 header

=====

Paragraphs are separated by a blank line.

2nd paragraph. **Italic**, ****bold****, and ``monospace``.
Itemized lists look like:

- * this one
- * that one
- * the other one

Sono formati testuali che usano trucchi testuali ad hoc per ottenere strutture e effetti tipografici precisi

Ottimi per testi lunghi, non permettono di catturare effetti tipografici specifici, forniscono supporto solo per le caratteristiche tipografiche più semplici.



JSON e YAML

JSON (Javascript Object Notation)

- Nato nei primi 2000 (Douglas Crockford).
- Formato dati testuale: oggetti (chiave-valore) e array;
- tipi base (string, number, boolean, null).
- Ottimo per API e storage.
- Non pensato per testo: solo stringhe grezze, nessuna semantica tipografica o strutturale (titoli, note, citazioni, stili), gestione scomoda di paragrafi lunghi o embedded markup.

YAML (Yet Another Markup Language)

- 2001 (Clark Evans), superset di JSON con sintassi ispirata a Python.
- L'indentazione è significativa, mapping/sequence, ancore e alias;
- Ottimo per file di configurazione e manifest.
- Non adatto al testo: wrapping e spaziature fragili, ambiguità sintattiche, assenza di semantica editoriale; rischioso per contenuti narrativi.

SGML

SGML (Standard Generalized Markup Language) è uno standard.

SGML è un meta-linguaggio non proprietario di markup descrittivo.

Facilita markup leggibili, generici, strutturali, gerarchici.



Linguaggio standard

SGML è uno standard ISO (International Standard Organization) n. 8879 del 1986. ISO è l'organizzazione mondiale degli standard, la più importante.

Essere uno standard per SGML significa che esso è il risultato di discussioni e compromessi di rappresentanti di tutte le comunità interessate, che è un investimento nel tempo di queste comunità, e che non dipende dai piani commerciali o dai capricci di una singola casa produttrice.



Meta-linguaggio di markup

Un meta-linguaggio è un linguaggio per definire linguaggi, una grammatica di costruzione di linguaggi.

SGML non è un linguaggio di markup, ma un linguaggio con cui definiamo linguaggi di markup.

SGML non dà dunque tutte le risposte di markup che possano sorgere a chi vuole arricchire testi per qualunque motivo, ma fornisce una sintassi per definire il linguaggio adatto.

SGML non sa cos'è un paragrafo, una lista, un titolo, ma fornisce una grammatica che ci permette di definirli.



Linguaggio non proprietario

Non proprietario significa che non esiste un'unica ditta o casa produttrice che ne detiene il controllo.

Viceversa RTF è © Microsoft, mentre PostScript e Acrobat sono © Adobe.

Essendo standard non proprietario, non dipende da un singolo programma, da una singola architettura. Gli stessi dati possono essere portate da un programma all'altro, da una piattaforma all'altra senza perdita di informazione.

Inoltre, anche l'evoluzione nel tempo è assicurata per lo stesso motivo.



Markup leggibile

In SGML il markup è posto in maniera leggibile a fianco degli elementi del testo a cui si riferiscono.

Essi possono sia essere usati da un programma, sia letti da un essere umano. Possono sia essere aggiunti da un programma (editor), sia da un essere umano con un programma non specifico.

Il markup in SGML è semplice testo facilmente interpretabile.



Markup descrittivo

Il markup in SGML non è pensato unicamente per la stampa su carta. E' possibile combinare markup utile per scopi o applicazioni diverse, ed in ogni contesto considerare o ignorare di volta in volta i markup non rilevanti.



Markup strutturato

SGML permette di definire delle strutture, suggerite o imposte, a cui i documenti si debbono adeguare.

Ad esempio, si può imporre che un testo sia diviso in capitoli, ognuno dei quali dotato di un titolo, una breve descrizione iniziale e almeno un paragrafo di contenuto.

È cioè possibile definire una serie di regole affinché il testo sia considerabile strutturalmente corretto.



Markup gerarchico

Le strutture imposte da SGML sono tipicamente a livelli di dettaglio successivi.

Gli elementi del testo possono comporsi gli uni con gli altri, permettendo di specificare la struttura in maniera gerarchica.

Ad esempio, si può imporre che il libro sia fatto di capitoli, e che questi a loro volta siano fatti di una descrizione e molti paragrafi. Una descrizione sarà quindi fatta di elementi, ciascuno dei quali sarà una frase composta di testo; i paragrafi saranno testo eventualmente contenenti anche elementi in evidenza o figure.



I documenti SGML

Un documento in un linguaggio di markup definito sulla base di SGML è sempre composto delle seguenti tre parti:

- Dichiarazione SGML
- Dichiarazione di documento (DOCTYPE)
- Istanza del documento



Un esempio di SGML

SGML
declaration

```
<!SGML "ISO 8879:1986" ...>
```

doctype

```
<!DOCTYPE novel [  
  <!ELEMENT novel (front,content)>  
  <!ELEMENT front (title, subtitle?, author)>  
  <!ELEMENT content (chapter+)>  
  <!ELEMENT chapter (title, para+)>  
  <!ATTLIST chapter id CDATA #REQUIRED>  
  <!ELEMENT title #PCDATA>  
  <!ELEMENT subtitle #PCDATA>  
  <!ELEMENT author #PCDATA>  
  <!ELEMENT para #PCDATA>  
>]
```

document instance

```
<novel>  
  <front>  
    <title>Three men in a boat</title>  
    <subtitle>To say nothing of the dog!</subtitle>  
    <author>Jerome K. Jerome</author>  
  </front>  
  <content>  
    <chapter id="c1">  
      <title>Chapter 1</title>  
      <para>There were four of us ... </para>  
      <para>We were all feeling ... </para>  
    </chapter>  
    <chapter id="c2">  
      ...  
    </chapter>  
  </content>  
</novel>
```



SGML Declaration

`<!SGML "ISO 8879:1986" car. spec.>`

La dichiarazione SGML contiene le istruzioni di partenza delle applicazioni SGML.

Essa permette di specificare valori fondamentali come la lunghezza dei nomi degli elementi, il set di caratteri usati, le specifiche caratteristiche di minimizzazione ammesse, ecc.)

Una dichiarazione SGML è lunga varie centinaia di righe.

Non è obbligatoria. Se è assente, viene usata una dichiarazione di default detta "Reference Concrete Syntax".

La RCS definisce lunghezze e sintassi standard (come l'uso del carattere "<" per indicare l'inizio del tag).



Document Type Declaration

`<!DOCTYPE nome [regole] >`

La dichiarazione del tipo del documento serve a specificare le regole che permettono di verificare la correttezza strutturale di un documento.

Vengono cioè elencati [i file che contengono] gli elementi ammissibili, il contesto in cui possono apparire, ed altri eventuali vincoli strutturali.

Nella terminologia SGML, si parla di modellare una classe (cioè una collezione omogenea) di documenti attribuendogli un tipo.



La document instance

L'istanza del documento è quella parte del documento che contiene il testo vero e proprio, dotato del markup appropriato.

Esso contiene una collezione di elementi (tag), attributi, entità, PCDATA, commenti, ecc.

Le applicazioni SGML sono in grado di verificare se l'istanza del documento segue le regole specificate nel DTD, e di identificare le violazioni.



I componenti del markup

Un documento con markup di derivazione SGML (inclusi HTML, XML, ecc.) contiene una varietà dei seguenti componenti

- Elementi
- Attributi
- Entità
- Testo (detto anche `#PCDATA`)
- Commenti
- Processing Instructions



Elementi, Attributi, Entità, #PCDATA, Commenti, Processing Instructions

Gli elementi sono le parti di documento dotate di un senso proprio.

Il titolo, l'autore, i paragrafi del documento sono tutti elementi.

Un elemento è individuato da un tag iniziale, un contenuto ed un tag finale.

Non confondere i tag con gli elementi!

`<titolo>Tre uomini in barca</titolo>`



Elementi, Attributi, Entità, #PCDATA, Commenti, Processing Instructions

Gli attributi sono informazioni aggiuntive sull'elemento che non fanno effettivamente parte del contenuto (meta-informazioni).

Essi sono posti dentro al tag iniziale dell'elemento. Tipicamente hanno la forma nome="valore"

```
<racconto tipo="romanzo">...</racconto>  
<capitolo id="c1">Capitolo primo</capitolo>
```



Elementi, Attributi, Entità, #PCDATA, Commenti, Processing Instructions

Le entità sono frammenti di documento memorizzati separatamente e richiamabili all'interno del documento. Esse permettono di riutilizzare lo stesso frammento in molte posizioni garantendo sempre l'esatta corrispondenza dei dati, e permettendo una loro modifica semplificata.

Oggi ` una bella giornata.
Come dice &FV;: "divertitevi!"
Quanta felicitÀ



Elementi, Attributi, Entità, #PCDATA, Commenti, Processing Instructions

Rappresenta il contenuto vero e proprio del documento. Esso corrisponde alle parole, gli spazi e la punteggiatura che costituiscono il testo.

Viene anche detto #PCDATA (Parsed Character DATA) perché i linguaggi di markup definiscono character data (CDATA) il contenuto testuale vero e proprio, e quello degli elementi è soggetto ad azione di parsing (perlopiù per identificare e sostituire le entità).



Elementi, Attributi, Entità, #PCDATA, Commenti, Processing Instructions

I documenti di markup possono contenere commenti, ovvero note da un autore all'altro, da un editore all'altro, ecc.

Queste note non fanno parte del contenuto del documento, e le applicazioni di markup li ignorano.

Sono molto comodi per passare informazioni tra un autore e l'altro, o per trattenere informazioni per se stessi, nel caso le dimenticassimo.

`<!-- Questo testo è ignorato dal parser -->`



Elementi, Attributi, Entità, #PCDATA, Commenti, Processing Instructions

Le processing instructions (PI) sono elementi particolari (spesso di senso esplicitamente procedurale) posti dall'autore o dall'applicazione per dare ulteriori indicazioni su come gestire il documento XML nel caso specifico

Per esempio, in generale è l'applicazione a decidere quando cambiare pagina. Ma in alcuni casi può essere importante specificare un comando di cambio pagina (oppure tutti i cambi pagina di un documento già impaginato).

<?newpage?>



XML 1.0

Una raccomandazione W3C del 10 febbraio 1998.

È definita come un sottoinsieme di SGML

Molto più formalizzata della grammatica di SGML, usa una notazione formale, Extended Backus-Naur Form.



Documenti ben formati o validi

XML distingue due tipi di documenti rilevanti per le applicazioni XML: i documenti ***ben formati*** ed i documenti ***validi***.

In SGML, un DTD è necessario per la validazione del documento. Anche in XML, un documento è **valido** se presenta un DTD ed è possibile validarlo usando il DTD.

Tuttavia XML permette anche documenti **ben formati**, ovvero documenti che, pur essendo privi di DTD, presentano una struttura sufficientemente regolare e comprensibile da poter essere controllata.



Documenti XML ben formati

Un documento XML si dice ben formato se:

- Tutti i tag di apertura e chiusura corrispondono e sono ben annidati
- Esiste un elemento radice che contiene tutti gli altri
- Gli elementi vuoti (senza contenuto) utilizzano un simbolo speciale di fine tag: `<vuoto/>`
- Tutti gli attributi sono sempre racchiusi tra virgolette
- Tutte le entità sono definite.



Conclusioni

Qui abbiamo parlato di

- Importanza del markup nel testo
- Un po' di storia del markup
 - Troff/Nroff
 - TeX e LaTeX
 - SGML – HTML
 - XML
 - JSON e YAML
 - Markdown e sintassi Wiki
- Cos'è SGML e di che cosa è fatto





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Corso di tecnologie web

Fabio.vitali@unibo.it

www.unibo.it